

Epsilon usage guide

author: *Jakub Liu*, date: *17.07.2026*

Before the start

Before we start the analysis, we need to check the following things.

1.) *Are all the software dependencies satisfied?*

Look at the list of dependencies at the bottom of this document.

2.) *Are all the bam files indexed and are all the index files in the same directory as the bam files themselves?*

3.) *Are all index files for the reference genome present?*

Assuming that our reference genome is names *hs37d5.fa* then the directory where the fasta file is located, these index files should also be present:

```
.
├── hs37d5.dict
├── hs37d5.fa
├── hs37d5.fa.amb
├── hs37d5.fa.ann
├── hs37d5.fa.bwt
├── hs37d5.fa.fai
├── hs37d5.fa.genome
├── hs37d5.fa.pac
└── hs37d5.fa.sa
```

4.) *Do we have the loci list csv file? And what type is it?*

- a. If we do not have a specific alternative allele whose presence we want to quantify, then our loci list file should look like this:

```
"chrom", "pos", "ref"
"9", 133748283, "C"
"9", 133738363, "G"
"14", 105246551, "C"
"2", 29443695, "G"
"2", 29432664, "C"
"X", 47426121, "C"
"7", 140453136, "A"
"7", 55259515, "T"
"7", 55249071, "C"
```

- b. If we have a specific alternative allele whose presence we want to quantify (e.g. in the case of calling a clinically actionable variant) then the loci list file should look like this:

```
"chrom", "pos", "ref", "alt"
"9", 133748283, "C", "T"
"9", 133738363, "G", "A"
"14", 105246551, "C", "T"
"2", 29443695, "G", "T"
"2", 29432664, "C", "T"
"X", 47426121, "C", "G"
"7", 140453136, "A", "T"
"7", 55259515, "T", "G"
"7", 55249071, "C", "T"
```

5.) Do the chromosome names in the reference genome, the bam files and the loci list file match?

For example, if the 4th chromosome in the reference genome is represented as 4, then it must be also denoted as 4 in both the bam files and the loci list file.

Finally, assuming that our project is named (very creatively) ‘project’, we need to make sure that our project directory structure looks like this:

```
(base) qbalu@Kruk:~$  
(base) qbalu@Kruk:~$ cd project/  
(base) qbalu@Kruk:~/project$ tree  
├── scripts  
│   ├── Epsilon_Fit.sh  
│   ├── Epsilon_MakeData.sh  
│   ├── Epsilon_MakeData_call.sh  
│   ├── Epsilon_call.sh  
│   ├── call.R  
│   ├── extract_features_alt_generic.py  
│   ├── extract_features_alt_generic_call.py  
│   ├── extract_features_alt_specific.py  
│   ├── extract_features_alt_specific_call.py  
│   ├── fit.R  
│   ├── make_data_alt_generic.py  
│   ├── make_data_alt_generic_call.py  
│   ├── make_data_alt_specific.py  
│   └── make_data_alt_specific_call.py
```

It is advised not to run any of the scripts from within the *scripts/* directory itself but from its parent directory (here the *projects/* directory). Okay, now having finished the boring stuff, let's try to call some mutations!

In the following sections I will try not digress about the statistics underlying the software implementation, for interested readers please have a look at the paper.

Step 1: creating the negative control cohort data

In the first step we need to extract the data from the negative control cohort bam files. The negative control cohort bam files are a set of normal files that are assumed to have no somatic variants. This can be achieved with the following command:

```
scripts/Epsilon_MakeData.sh \  
  --bamlist negative_control_bamlist.txt \  
  --loci_list Civic_actionable_SNV_loci.csv \  
  --reference_genome hs37d5.fa \  
  --alt_mode generic \  
  --nranks 16 \  
  --output_prefix NEGATIVE_CONTROL_DATA3
```

--bamlist

A .txt file with paths to the bam files from the negative control cohort. One path per line.

--loci_list

A .csv file of the form:

```
"chrom", "pos", "ref"
"9", 133748283, "C"
"9", 133738363, "G"
"14", 105246551, "C"
"2", 29443695, "G"
"2", 29432664, "C"
"X", 47426121, "C"
"7", 140453136, "A"
"7", 55259515, "T"
"7", 55249071, "C"
```

If we have no specific alternative allele, or:

```
"chrom", "pos", "ref", "alt"
"9", 133748283, "C", "T"
"9", 133738363, "G", "A"
"14", 105246551, "C", "T"
"2", 29443695, "G", "T"
"2", 29432664, "C", "T"
"X", 47426121, "C", "G"
"7", 140453136, "A", "T"
"7", 55259515, "T", "G"
"7", 55249071, "C", "T"
```

If we have a specific alternative allele.

`--reference_genome`

The reference genome

`--alt_mode [generic | specific]`

The mode must match the specified `-loci_list` file.

`--nranks [int]`

The number of parallel processes to be run.

`--output_prefix`

The prefix to the output file.

After a successful run, the negative control data will be found in the *negative_control_data/* directory. The directory structure will look like this:

```
.
├── negative_control_data
│   └── negative_control_data.csv
├── scripts
│   ├── Epsilon_Fit.sh
│   ├── Epsilon_MakeData.sh
│   ├── Epsilon_MakeData_call.sh
│   ├── Epsilon_call.sh
│   ├── call.R
│   ├── extract_features_alt_generic.py
│   ├── extract_features_alt_generic_call.py
│   ├── extract_features_alt_specific.py
│   ├── extract_features_alt_specific_call.py
│   ├── fit.R
│   ├── make_data_alt_generic.py
│   ├── make_data_alt_generic_call.py
│   ├── make_data_alt_specific.py
│   └── make_data_alt_specific_call.py
```

Step2: fitting the error model

The next step is to fit a statistical model to the created negative control data. This can be done with the following command:

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
"$SCRIPT_DIR/scripts/Epsilon_Fit.sh" \  
  --input negative_control_data.txt \  
  --noise_level 0.01 \  
  --alt_mode generic \  
  --output_model alt_generic_model
```

--input

The name of the negative control dataset.

--noise_level [int [0-1]]

The maximum value of the variant allele fraction that is allowed in the negative control dataset. A lower value implies in theory a higher sensitivity at the cost of a lower specificity. A higher value implies in theory the opposite.

--alt_mode

The mode must match the mode from **Step 1**.

--output_model

The name of the saved .rds file

After a successful run the model (and the optional scaling .rds file (details intentionally omitted here)) will be found in the *fitted_error_model/* directory.

```
.  
├── fitted_error_model  
│   ├── model.rds  
│   └── scaling_parameters.rds  
├── negative_control_data  
│   └── negative_control_data.csv  
└── scripts  
    ├── Epsilon_Fit.sh  
    ├── Epsilon_MakeData.sh  
    ├── Epsilon_MakeData_call.sh  
    ├── Epsilon_call.sh  
    ├── call.R  
    ├── extract_features_alt_generic.py  
    ├── extract_features_alt_generic_call.py  
    ├── extract_features_alt_specific.py  
    ├── extract_features_alt_specific_call.py  
    ├── fit.R  
    ├── make_data_alt_generic.py  
    ├── make_data_alt_generic_call.py  
    ├── make_data_alt_specific.py  
    └── make_data_alt_specific_call.py
```

Step 3: Creating the data from the tumor samples

Now we need to create data from the tumor sample(s) in which we want to quantify the presence of a mutation that is analogous to the negative control data from step **Step 1**.

```
scripts/Epsilon_MakeData_call.sh \  
  --bamlist tumor_bam.txt \  
  --loci_list calling.bed.csv \  
  --reference_genome hs37d5.fa \  
  --alt_mode generic \  
  --nranks 8 \  
  --output_prefix tumor_data
```

--bamlist

A .txt file of paths to tumor bam files, one path per line. Even if we have a single bam file, then we need to wrap it in a .txt file.

--loci_list

The list of loci in which we want to potentially find the variants. It is analogous to the *--loci_list* from **step 1**.

--reference_genome

The reference genome

--alt_mode

Must be the same as the **Step 1. And 2**.

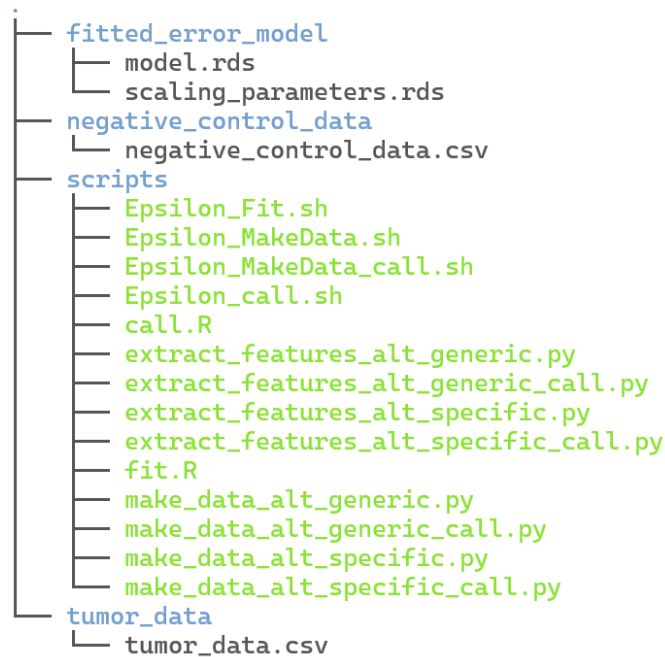
--nranks

The number of parallel processes.

--output_prefix

The prefix of the output file.

After a successful run the output file is located in the *tumor_data/* directory.



Step 4: Variant calling

The final step is to quantify the presence of mutations in the data. Here we have two main options. We can call variants using the Binomial mode or the Bayesian mode. If the Binomial mode is chosen, then we can further specify the significance level and the multiple testing correction. If the Bayesian mode is chosen, then we can specify the prior and the posterior threshold. Both modes can be run the specific and generic mode. More details can be found in the paper. In the absence of specific reasons, I recommend to use the Binomial mode. The recommended script call is shown below:

```

"$SCRIPT_DIR/scripts/Epsilon_call.sh" \
  --model fitted_error_model/model.rds \
  --input tumor_data/tumor_data.txt \
  --output generic.binomial.vcf \
  --mode binomial_test \
  --fdr_method BH \
  --alpha 0.05 \
  --alt_mode generic

```

--model

The fitted error model (from step 2.).

--input

The tumor data (from step 3.)

--output

The output vcf file.

--mode

The statistical variant calling mode. *binomial_test* or *bayes_posterior*.

--fdr_method

The multiple testing correction method. Available options: *holm*, *Hochberg*, *hommer*, *bonferroni*, *BH*, *BY*, *fdr*.

--alpha

The significance level.

--alt_mode

Must match the mode in the previous steps.

Below I show the Bayesian variant calling mode. Only the parameters that differ from the previous listing are described.

```
"$SCRIPT_DIR/scripts/Epsilon_call.sh" \  
  --model fitted_error_model/model.rds \  
  --input tumor_data/tumor_data.txt \  
  --output generic.bayes.vcf \  
  --mode bayes_posterior \  
  --prior 0.3 \  
  --posterior_cutoff 0.5 \  
  --alt_mode generic
```

--prior

The prior used. This is one of the hyperparameters that you can use to leverage the sensitivity and specificity. For more information on the usage of the prior please read the paper.

--posterior_cutoff

A posterior above this cutoff will lead to a variant call. This is the second hyperparameter that can be used to manipulate sensitivity and specificity.